ARC-SPEC-2026-EDU-07: Pedagogical Cryptography, Zero-Knowledge Verification & Sovereign Identity Invariants

A Zero-Knowledge Proof Architecture for Asynchronous Assessment, Decentralized Credential Attestation, and Privacy-Preserving Academic Verifiability

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Knowledge Leakage ($\mathcal{K}_{\text{leak}}$) $= 0$ bits
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Zero-Knowledge Pedagogy Rationale

ARC-SPEC-2026-EDU-01 through EDU-06 built the complete physical, mathematical, educational, and verification foundation for Deterministic Admissibility Engineering (DAE). However, a critical friction point remains in decentralized academic attestation: evaluating student competence and issuing industrial credentials without compromising proprietary code, exposing sensitive student data, or introducing central authority verification drag.

Traditional educational credentialing relies on centralized, high-drag institutions that violate $C_{\text{ops}} = 0$ and introduce privacy vulnerabilities. Furthermore, in commercial or defense settings, engineers completing DAE laboratories often work on proprietary bare-metal substrates where sharing raw source code or test logs publicly is strictly forbidden.

This specification formalizes ARC-SPEC-2026-EDU-07: a Zero-Knowledge Pedagogical Protocol (ZKPP). Using zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) and Decentralized Identifiers (DIDs), EDU-07 enables an engineer to mathematically prove that their implementation satisfies all DAE invariants ($C_{\text{ops}} = 0$, $\tau_{\text{override}} \leq 11.99\text{ms}$, Zero-Heap) without revealing a single line of underlying source code.

Credential Verification Comparison

TRADITIONAL CREDENTIAL VERIFICATION (High Leakage / Central Drag):

[Student Code / Transcripts] $\longrightarrow$ [Central Registrar / Employer] $\longrightarrow$ [Manual Audit]

ZKPP ZERO-KNOWLEDGE ATTESTATION (ARC-SPEC-2026-EDU-07 / Zero Drag):

$$\text{[Private Code Implementations]} \longrightarrow \text{[Local zk-SNARK Prover]} \longrightarrow \text{[Proof } \pi_{\text{DAE}}\text{]}$$
$$\Downarrow$$
$$\longrightarrow \text{[Instant Verifier Pass]} \text{ (Zero Code Leakage)}$$

2 Circom Circuit Implementation (`dae_verifier.circom`)

The zero-knowledge verification circuit is archived under `/docs/academic/zk/dae_verifier.circom`. The Circom code below enforces execution admissibility constraints in zero-knowledge.

```
pragma circom 2.1.6;

template DAEAdmissibilityVerifier() {
    // 1. PUBLIC INPUTS (Visible to Verifier)
    signal input publicCommitmentHash;
    signal input maxAllowedCycles;
    signal input targetPosixCode;

    // 2. PRIVATE WITNESS (Kept secret on student's local node)
    signal input privateSourceCodeHash;
    signal input privateMeasuredCycles;
    signal input privateHeapAllocBytes;
    signal input privateEvaluatedPosixCode;

    // 3. CIRCUIT CONSTRAINTS
    // Constraint A: Zero Heap Rule (MUST BE 0)
    privateHeapAllocBytes === 0;

    // Constraint B: Latency Bound Check
    component cycleCheck = LessEqThan(64);
    cycleCheck.in[0] <== privateMeasuredCycles;
    cycleCheck.in[1] <== maxAllowedCycles;
    cycleCheck.out === 1;

    // Constraint C: Poset Lattice Resolution
    privateEvaluatedPosixCode === targetPosixCode;
}

component main {public [publicCommitmentHash, maxAllowedCycles,
   targetPosixCode]} = DAEAdmissibilityVerifier();
```

Listing 1: `dae_verifier.circom` Admissibility Verifier Circuit

3 Zero-Knowledge Proving and Verification Pipeline

To preserve $C_{\text{ops}} = 0$, the student executes proof generation locally on their bare-metal development node via a single CLI command.

```
# 1. Local SNARK Proof Generation Execution Command
snarkjs groth16 prove dae_verifier_final.zkey witness.wtns proof.json
   public.json

# 2. Zero-Knowledge Verification Command (Executed by CI)
snarkjs groth16 verify verification_key.json public.json proof.json
```

Listing 2: Groth16 Zero-Knowledge Verification Commands

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-07
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

Table 1: ZKPP Circuit Execution Performance Bounds

Phase	**Execution Substrate**	**Performance Invariant Ceiling**
Witness Generation	Local Developer Node	Complete execution trace generation ≤ 1.8s
Proof Generation (π_{DAE})	Groth16 Prover	Proof size exactly 128 bytes ($\mathcal{K}_{\text{leak}} = 0$ bits)
Verification	`substrate-ci.yml` / Smart Contract	Verification time ≤ 1.2ms ($C_{\text{ops}} = 0$)